Informe de la práctica 11

Persistencia local y estrategia offline first

Estudiante: Grease Duran Castro

Asignatura: Desarrollo de Aplicaciones Móviles

Docente: Mg. Reyna Barreto Benjamin David

Fecha: 20 de octubre de 2026

Proyectos: PharmaMobil y PharmaSoft

Rama: feature/offline-first-duran

1 Objetivo

Convertimos el inventario de PharmaMobil en una pantalla que funciona desde una base de datos local. El listado ya no depende de una respuesta de PharmaSoft para mostrarse. Los productos creados sin red se conservan como pendientes y se envían al recuperar la conexión.

2 Resultado obtenido

La aplicación abrió en modo avión, mostró el catálogo guardado y permitió crear un producto. Ese registro sobrevivió al cierre y reapertura, y luego llegó al servidor con un identificador positivo. También se comprobó el arranque con PharmaSoft apagado y la red del emulador habilitada.

Comprobación	Resultado
Compilación Android	BUILD SUCCESSFUL
Pruebas automatizadas	90 aprobadas; 0 fallos, errores u omisiones
Pruebas nuevas de persistencia y sesión offline	13 aprobadas
Producto de demostración	S11Offline-1020143628
Antes de sincronizar	ID local -1 y pendiente 1
Después de sincronizar	ID del servidor 82 y pendiente 0
Consulta en Swagger	GET productos/82 responde 200
iOS	Driver preparado; ejecución pendiente en Mac

3 Implementación realizada

Esquema y drivers

Se agregó SQLDelight 2.1.0. Producto.sq define las consultas y los campos id, nombre, precio, stock y pendiente. Se conservaron estado y categoría para el CRUD anterior, y se añadió usuario para separar las filas de cada cuenta. La clase PharmaDb se genera durante la compilación. Android abre pharma.db con AndroidSqliteDriver; iOS utiliza NativeSqliteDriver. Las pruebas comunes cuentan con una fábrica de driver y cada caso crea una base independiente.

Fuente de verdad local

ProductoLocal convierte las filas SQL al modelo de dominio y expone consultas observables mediante Flow. Esta clase no importa Ktor ni DTO. ProductoRepositorioOffline ofrece observar y sincronizar; los detalles de producto también se leen localmente. El ViewModel recoge el Flow una vez. Crear, editar y eliminar modifican SQLite y actualizan la pantalla sin recargar manualmente una lista remota.

Sincronización de pendientes

Primero se envían las filas pendientes y después se descarga el catálogo paginado. Cada creación usa un ID negativo hasta recibir el identificador real. Una subida confirmada se registra de inmediato; si falla la descarga posterior, el siguiente intento no repite ese POST. Un mutex evita sincronizaciones simultáneas. La bajada reemplaza solo filas sincronizadas dentro de una transacción y no borra trabajo pendiente.

Edición y eliminación

Una edición pendiente de una fila positiva se envía con PUT. Una eliminación se guarda como fila inactiva pendiente hasta completar DELETE. Eliminar una creación que aún tiene ID negativo cancela esa fila local sin enviarla al servidor. El contador contempla también eliminaciones pendientes que ya no aparecen en el listado activo.

Sesión y límites

El JWT permanece en el almacén cifrado de la sesión 10. Para abrir sin red se conservan usuario, nombre y permisos de una sesión previamente validada. Solo se recupera con un token aún vigente y ante un fallo de conexión o timeout. Una respuesta 401 del servidor no utiliza esta alternativa: limpia la sesión. El primer login y una sesión vencida requieren red. La autorización de cada petición continúa en PharmaSoft. iOS no se ejecutó por falta de Mac con Xcode.

4 Catálogo con red y sin conexión

La primera captura corresponde al catálogo sincronizado. Después se cerró la aplicación, se activó modo avión y se desactivaron Wi-Fi y datos móviles. Al reabrir, el listado apareció desde SQLite.

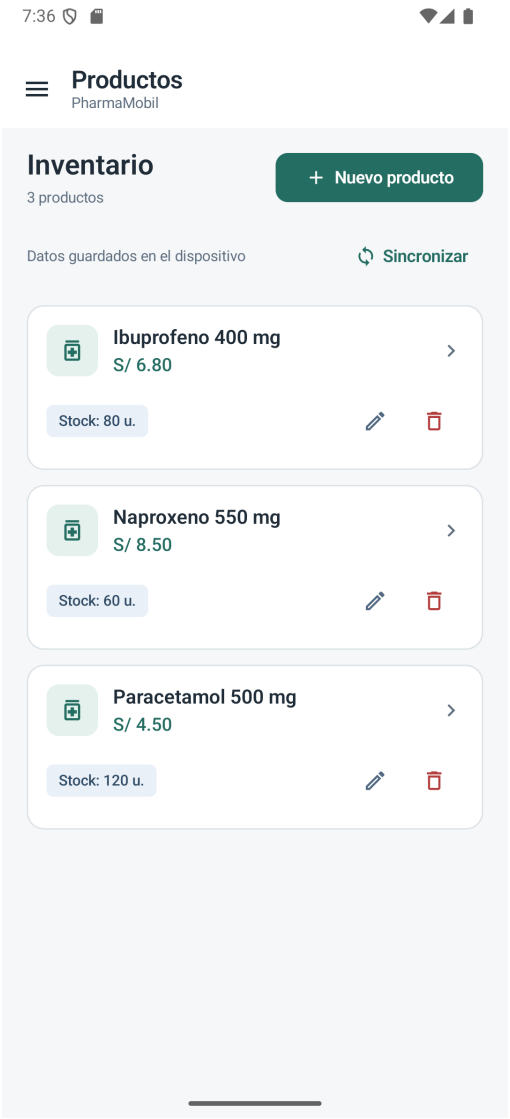


Figura 1. Catálogo con red, tres productos y botón Sincronizar.

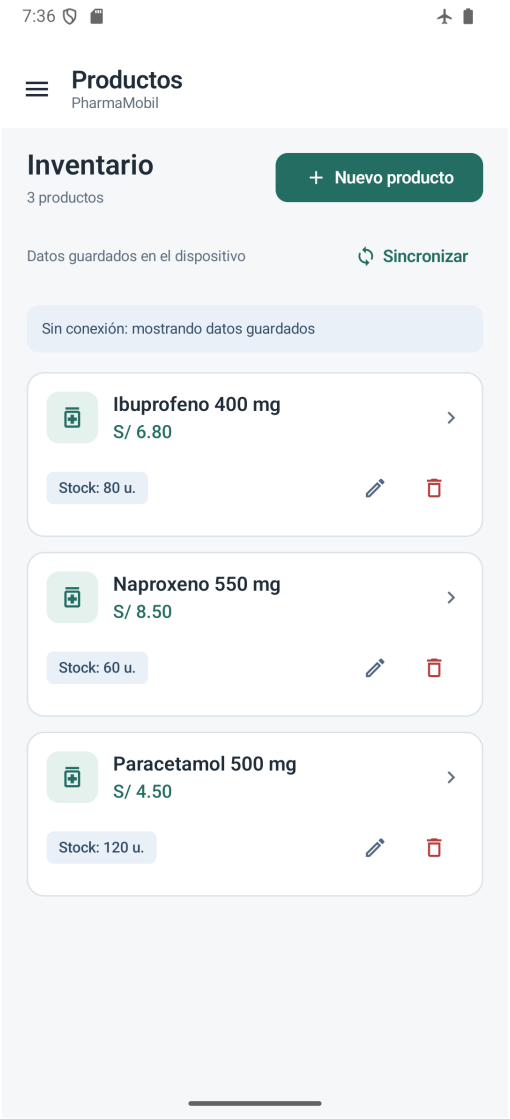


Figura 2. Arranque en modo avión con los mismos productos y aviso sin conexión.

5 Creación y persistencia del pendiente

Se registró S11Offline-1020143628 con precio 5.70 y stock 12. La revisión de SQLite registró id -1 y pendiente 1. Se cerró completamente la app y se volvió a abrir sin conexión.

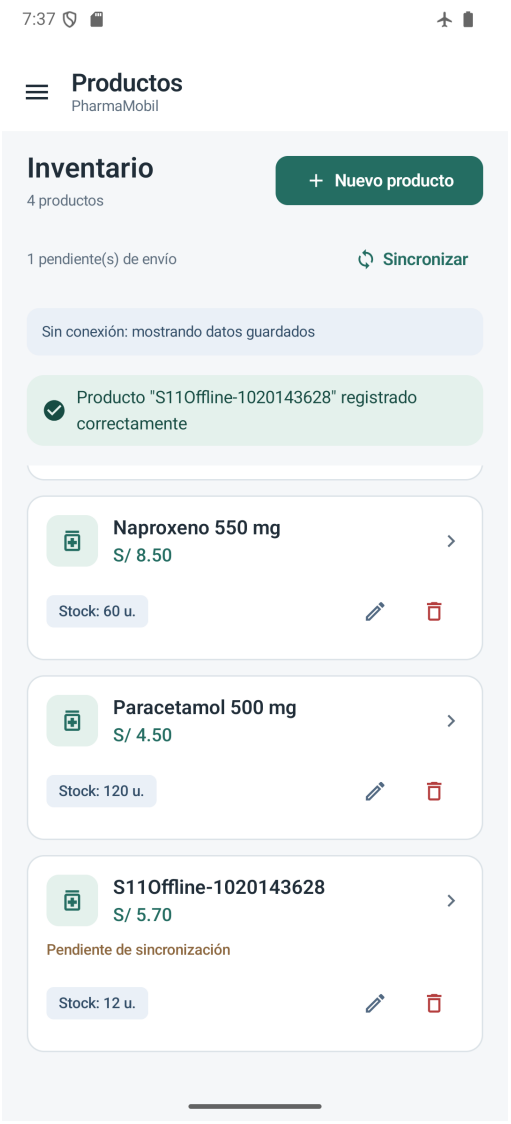


Figura 3. Producto creado sin red y marcado pendiente de sincronización.

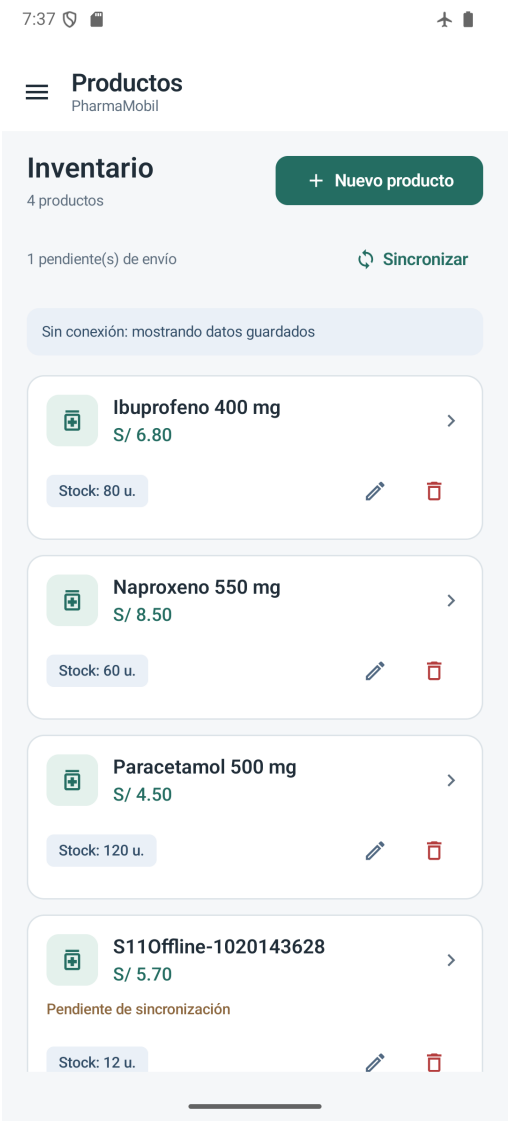


Figura 4. El producto continúa presente y pendiente después de reabrir la app.

6 Reconexión y backend apagado

Al recuperar Wi-Fi y datos se pulsó Sincronizar. La fila local pasó de ID -1 a 82 y su marca pendiente se eliminó. Después se detuvo PharmaSoft y se reabrió la app con red habilitada para comprobar la lectura local.

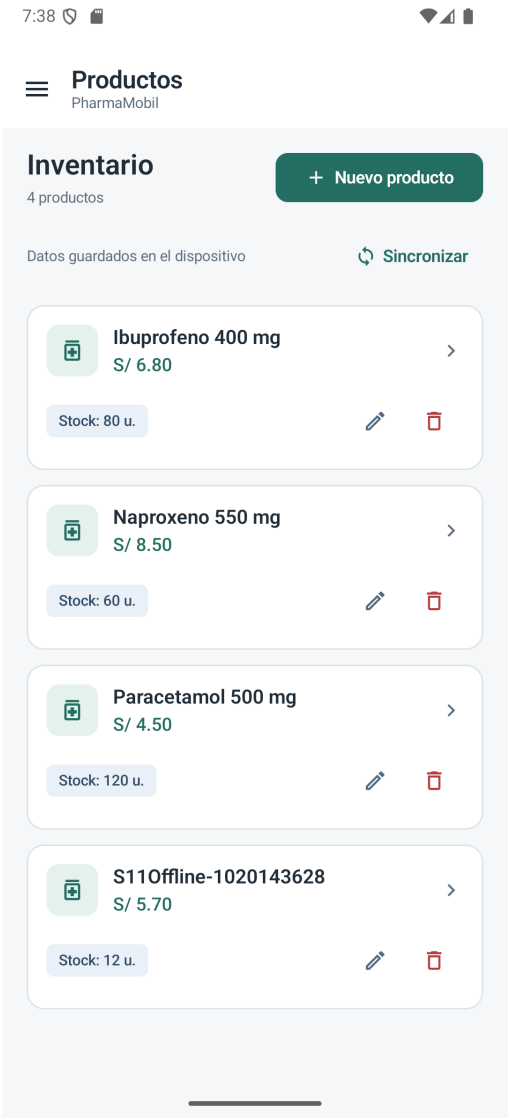


Figura 5. Producto sincronizado; ya no aparece como pendiente.

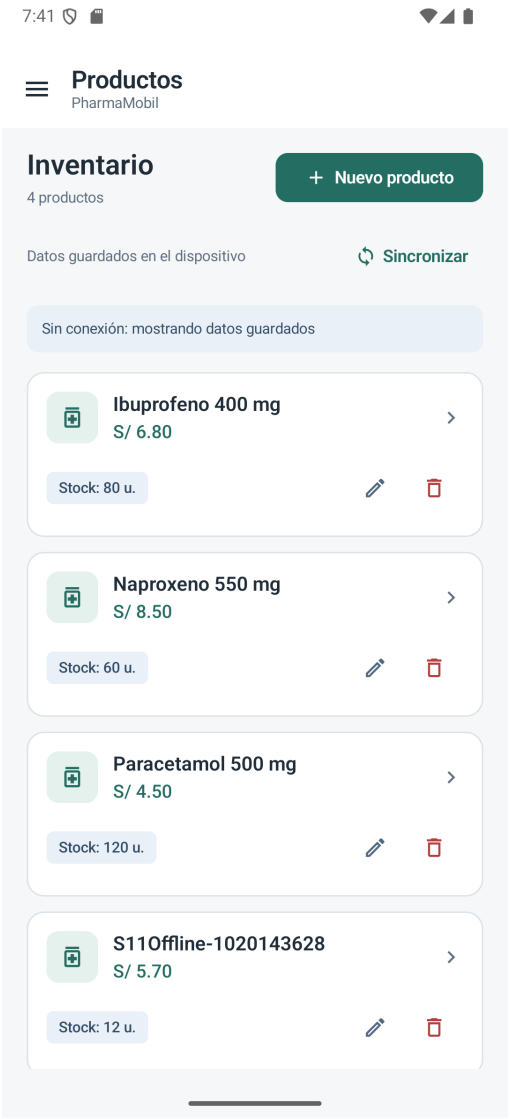


Figura 6. Aplicación abierta con PharmaSoft apagado; el inventario sigue disponible.

7 Confirmación del producto en PharmaSoft

Swagger consultó GET /api/v1/productos/82 con autenticación de admin. La respuesta real fue 200 y contiene el mismo nombre, precio 5.70, stock 12 e ID positivo. Esta comprobación acredita que el producto salió del dispositivo y existe en el servidor. El encuadre omite el JWT que Swagger incluye en el ejemplo curl.

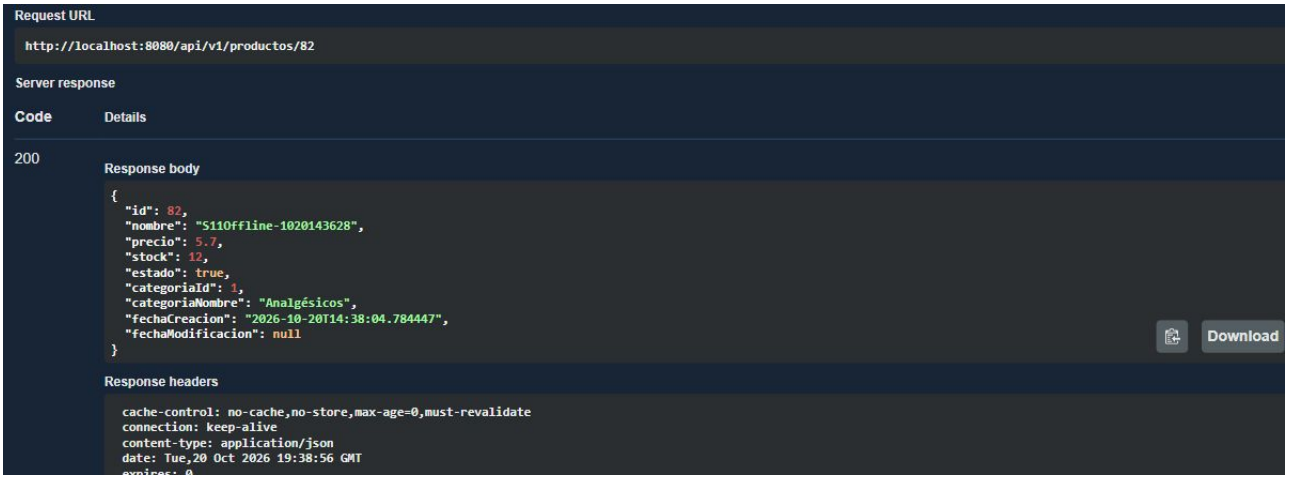


Figura 7. URL, código 200 y cuerpo real del producto confirmado.

8 Pruebas y registros

Las 13 pruebas nuevas cubren creación sin red, orden POST antes de GET, confirmación antes de una descarga fallida, conservación de pendientes, aislamiento por usuario, edición y eliminación local, reapertura del repositorio, sincronizaciones simultáneas, eliminación confirmada seguida de error 500, recuperación de sesión vigente sin red y rechazo de una restauración ante 401 o token vencido.

Se conservaron las pruebas anteriores; el total final es 90. La salida real se encuentra en docs/SESION11/compilacion-y-pruebas.log. pruebas.json contiene el resumen y los nombres de los casos nuevos. bitacora-android.json guarda acciones, estados de conexión y filas inspeccionadas. producto-confirmado-servidor.json conserva la respuesta de la API y verificacion.json identifica el APK mediante SHA-256.

Las capturas se conservan sin modificar. La bitácora utiliza hora de Lima; el reloj visible del emulador tiene otra zona horaria.

9 Pasos para reproducir

1. Encender Oracle Docker y PharmaSoft. Abrir PharmaMobil en Android Studio, compilar y ejecutar androidApp en el emulador.

2. Iniciar sesión con admin y contraseña Admin123. Entrar a Productos y pulsar Sincronizar para guardar el catálogo.

3. Cerrar por completo la aplicación. Activar modo avión y desactivar Wi-Fi y datos. Reabrir y verificar los productos y el aviso sin conexión.

4. Registrar un producto de prueba y comprobar su marca pendiente. Cerrar y reabrir para verificar que persiste.

5. Recuperar la conexión y pulsar Sincronizar. Consultar el producto en Swagger: debe tener ID positivo y no debe duplicarse.

6. Detener PharmaSoft sin apagar Docker. Reabrir la aplicación y comprobar que conserva el catálogo. Finalmente volver a iniciar PharmaSoft.

7. Ejecutar verificar-sesion11.ps1 desde PowerShell para compilar el APK y ejecutar las pruebas automatizadas.

10 Conclusiones

El inventario funciona con la base local como fuente de verdad. El ciclo real comprobó el paso de un registro pendiente con ID negativo a un producto confirmado en PharmaSoft. La sesión y los datos locales permiten reabrir sin red mientras el token siga vigente. Las pruebas automatizadas y las capturas respaldan el funcionamiento en Android.

La ejecución iOS continúa pendiente. Tampoco se implementa resolución automática de conflictos entre dispositivos. Si se pierde la respuesta de un POST después de que el servidor haya creado el producto, el backend no ofrece una clave de idempotencia y no puede garantizarse evitar un duplicado en ese caso ambiguo. La confirmación por fila sí evita repetir una creación cuando falla una descarga posterior.